Encoding and Embedding

One Hot Encoding (word presence) → "Is the word present or not?"

Bag of Words (word count) → "How many times does the word appear?"

TF-IDF (count + importance weighting) → "How important is the word in the document?" → It reduces the weight of common words and increases the weight of more informative words

Embeddings (semantic + contextual understanding) → "What is the meaning of the text in the context (words, sentence, paragraph, corpus)?"

One Hot Encoding

D1 → people watch movie
D2 → people watch football
D3 → people like movie
D4 → people like football

vocabulary : {people, watch, like, movie, football}

	people	watch	like	movie	football
D1 →	1	1	0	1	0
D2 →	1	1	0	0	1
D3 →	1	0	1	1	0
D4 →	1	0	1	0	1

<u>Document level representation</u>
D1 → people watch movie → [1, 1, 0, 1, 0]
D2 → people watch football → [1, 1, 0, 0, 1]
D3 → people like movie → [1, 0, 1, 1, 0]
D4 → people like football → [1, 0, 1, 0, 1]

<u>word level (sequence) representation</u>
D1 → people watch movie → [
[1, 0, 0, 0, 0]
[0, 1, 0, 0, 0]
[0, 0, 0, 1, 0]
]

D2 → people watch football → [
[1, 0, 0, 0, 0]
[0, 1, 0, 0, 0]
[0, 0, 0, 0, 1]
]

D3 → people like movie → [
[1, 0, 0, 0, 0]
[0, 0, 1, 0, 0]
[0, 0, 0, 1, 0]
]

D4 → people like football → [
[1, 0, 0, 0, 0]
[0, 0, 1, 0, 0]
[0, 0, 0, 0, 1]
]

Pros
1. Easy to implement
2. Simple binary representation
3. No training required
4. Direct mapping

Cons
1. Sparse matrix mostly zeros → memory waste
2. Sparse matrix means huge vector size but very little useful information
3. High dimensionality vocabulary increases → vector size increases
4. No semantic understanding, similar words also seems unrelated
5. OOV problem could not handle new words

Bag of Words

Bag of words captures how frequently a word is used in a meaningful context.
"If a words repeats more in a sentence, BoW assumes it is more important."

D1 → people watch movie and watch movie again
D2 → people watch football and watch football
D3 → people like movie and like movie a lot
D4 → people like football

vocabulary : {again, and, football, like, lot, movie, people, watch}

	again	and	football	like	lot	movie	people	watch
D1 →	1	1	0	0	0	2	1	2
D2 →	0	1	2	0	0	0	1	2
D3 →	0	1	0	2	1	2	1	0
D4 →	0	0	1	1	0	0	1	0

Document level representation

D1 → people watch movie and watch movie again → [1, 1, 0, 0, 0, 2, 1, 2]
D2 → people watch football and watch football → [0, 1, 2, 0, 0, 0, 1, 2]
D3 → people like movie and like movie a lot → [0, 1, 0, 2, 1, 2, 1, 0]
D4 → people like football → [0, 0, 1, 1, 0, 0, 1, 0]

Pros

1. Easy to implement
• very simple logic
• just count word occurrences
• no complex math required

2. Captures word frequency
• shows how often a word appears
• more occurrences → higher importance

3. Works well before embedding for classical NLP tasks
• useful for text classification, spam detection and basic level sentiment analysis

4. No training required
• no learning phase
• direct conversion from text → numbers

Cons

1. Ignores word order
• Sentence meaning can change, BoW won't notice
• Example:
  • "dog bites man" and "man bites dog", has same BoW representation

2. No semantic understanding
• Similar words are treated completely different
• Example:
  • "like" vs "love", has no relation captured

3. High dimensionality
• Vocabulary size = vector size
• Large dataset → huge vectors

4. Sparse representation
• Mostly values are 0
• Memory inefficient
• Computation waste

5. OOV (Out of Vocabulary) problem
• New words cannot be handled

6. Overemphasizes frequent words
• Example:
  • "movie movie movie", gets high importance even if not meaningful

---

TF-IDF (Term Frequency - Inverse Document Frequency)

D1 → people watch football
D2 → football watch football
D3 → people give comment
D4 → football give comment

vocabulary : {comment, football, give, people, watch}

TF(word, D) = occurrences of the word in a sentence / total words in a given sentence

IDF(word) = log (total number of sentence) / number of sentence containing the word

		comment	**football**	**give**	**people**	**watch**
D1 →	people watch football	0	(1/3) * log(4/3)	0	(1/3) * log(4/2)	(1/3) * log(4/2)
D2 →	football watch football	0	(2/3) * log(4/3)	0	0	(1/3) * log(4/2)
D3 →	people give comment	(1/3) * log(4/2)	0	(1/3) * log(4/2)	(1/3) * log(4/2)	0
D4 →	football give comment	(1/3) * log(4/2)	(1/3) * log(4/3)	(1/3) * log(4/2)	0	0

		comment	**football**	**give**	**people**	**watch**
D1 →	people watch football	0	0.096	0	0.231	0.231
D2 →	football watch football	0	0.191	0	0	0.231
D3 →	people give comment	0.231	0	0.231	0.231	0
D4 →	football give comment	0.231	0.096	0.231	0	0

Why TF (Term Frequency)?
Because more occurrences = more important in the document

Why IDF (Inverse Document Frequency)?
Because common words are not useful

Why multiply TF * IDF?
Because, TF → importance in document, IDF → Importance in corpus

Why log in IDF?
Without log, rare words get extremely high values

Log:
smooths values and keeps balance
log reduces extreme differences while keeping the importance order intact.
log compresses large values so rare words don't overpower everything

Pros
1. Captures importance of words
rare words → higher weight
common words → lower weight

2. Reduces impact of common words
IDF reduces their weight

3. Better than BoW for relevance and helps in
Search engines
information retrieval
ranking documents

4. No training required
Direct calculation
No model learning

5. Simple and interpretable
Easy to understand mathematically
TF * IDF → clear logic

Cons
1. Ignores word word
Example:
"dog bites man" and "man bites dog", has same BoW representation

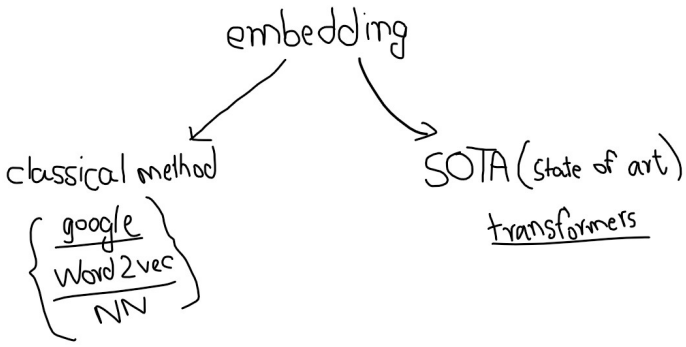
2. No semantic understanding
Cannot understand meaning
Example:
"car" and "automobile", treated as completely different

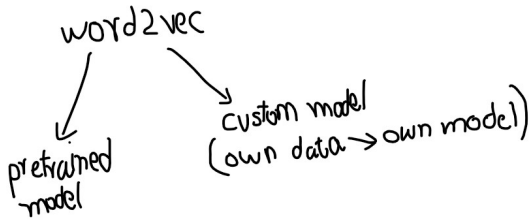
3. Still sparse and high dimensional
Vocabulary size = vector size
Many zeros → inefficient

4. Does not handle context
same word, different meaning → same vector
"bank" (river vs finance)

5. OOV problem
New words not in vocabulary → ignored

Word2Vec





Sentence 1: I like this movie

Sentence 2: I love this film

For OHE / BoW / TF-IDF	Embeddings
like ≠ love	like ≈ love
movie ≠ film	movie ≈ film

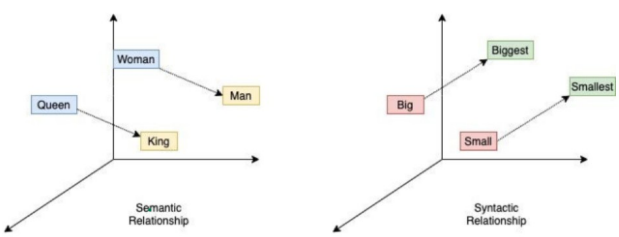
word2vec → Neural Network
↳ google → 2013

300 Features
↓
word → (300 dimensions)

Agenda → Word → Number (vector)
↳ Feature ← model

Word2vec
↙ CBOW
↘ Skipgram

	Man	Woman	King	Queen	Apple	Orange
Royal	0.01	0.02	0.93	0.95	-0.01	0.00
Age	0.03	0.02	0.7	0.69	0.03	-0.02
Food	0.04	0.01	0.02	0.01	0.95	0.97



King − Man + Woman = Queen

Vocab

I love AI and I love coding
↳ set of unique words {I, love, AI, and, coding}

Joe love AI and joe love coding
↳ {Joe, love, AI, and, joe, coding}

} Case Sensitive
Apple ≠ apple

Window size

I love deep learning

window size ↑ context ↑
very high window size ↑ noise ↑

size = 1

Context	Target
love	I
I, deep	love
love, learning	deep
deep	learning

size = 2

Context	Target
love, deep	I
I, deep, learning	love
I, love, learning	deep
love, deep	learning

Word2vec

CBOW

Skip-gram

I love deep learning

Context	Target
love	I
I, deep	love
love, learning	deep
deep	learning

Context	Target
I	love
love	I, deep
deep	love, learning
learning	deep

Step 1 I love deep learning

→ Vocab {I, love, deep, learning}

Step 2

Step 3 OHE

I → [1 0 0 0]

love → [0 1 0 0]

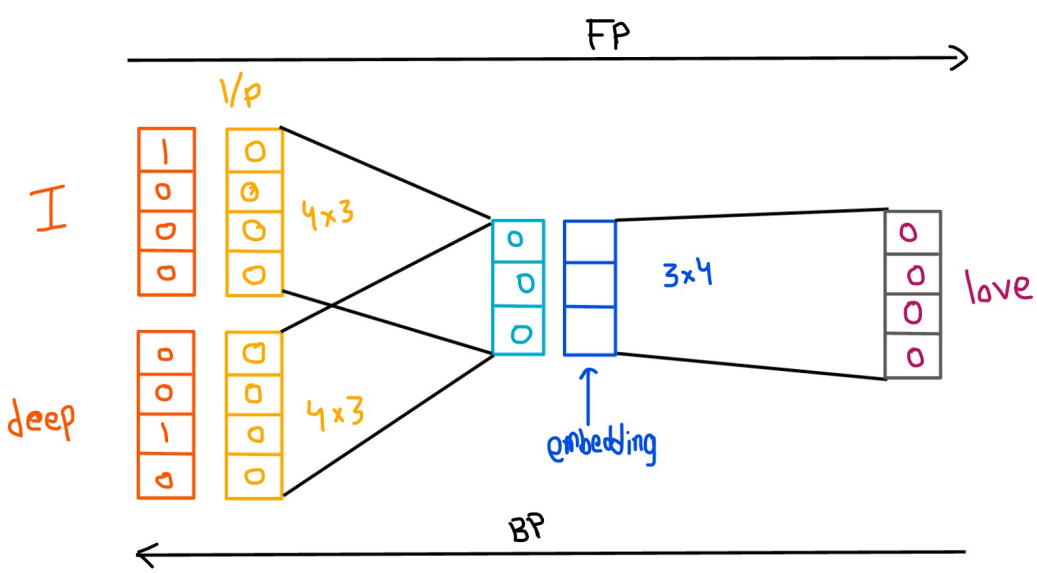
deep → [0 0 1 0]

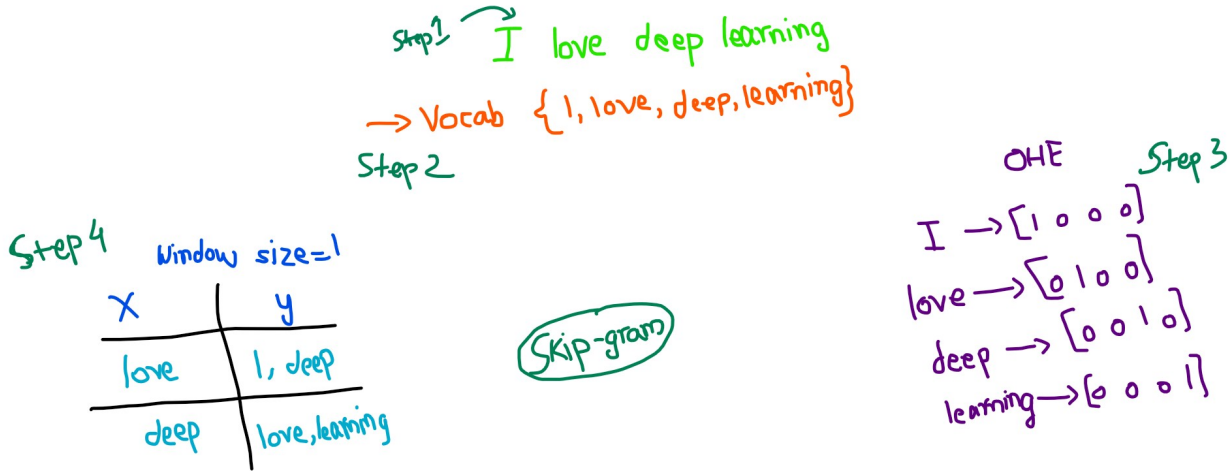
learning → [0 0 0 1]

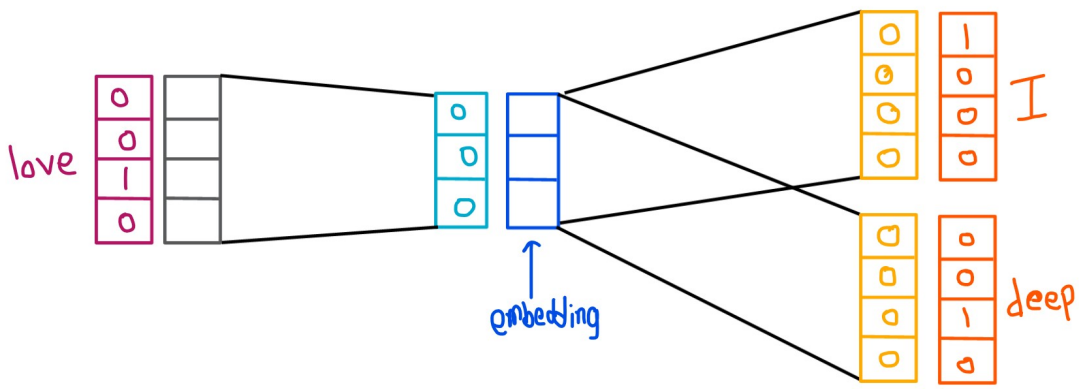
Step 4 Window size=1

X	y
I, deep	love
love, learning	deep

CBOW







Comparison of CBOW and Skip Gram

Feature	**CBOW**	**Skip-gram**
Speed	Fast	Slow
Accuracy	Medium	High
Rare words	Weak	Strong
Data requirement	Less	More
Part		
Input vectors	Multiple	Single
Hidden layer	Average	Direct
Output	Single	Multiple

CBOW and skip-gram differ not only in data preparation but also in objective function, training dynamics, and learning behavior.

CBOW predicts a target word from the surrounding context using averaged embeddings, making it faster but less effective for rare words.

Skip-gram predicts multiple context words from a single target word, resulting in better representation, especially for rare words, but with higher computational cost.

Pros

1. Efficient and fast training
- Shallow model (no deep layers)

2. Semantic understanding
- Similar words → close vectors
- Example
  - king ≈ queen
  - car ≈ vehicle

3. Vector arithmetic
- queen ≈ king - man + woman

4. Pretrained embeddings available
- Google News vectors (300 dimensions)
- Easy to reuse

5. General purpose methodology
- Can be used in:
  - NLP pipelines
  - ML models
  - Clustering / search

Cons

1. Context independent
- Same word = same vector
  - bank (river) = bank (finance)

2. Fixed embeddings
- Vector would be static
- will not change according to the sentence context

3. No handling of OOV
- New word is unknown

4. Cannot handle sub-words
- Words like:
  - running, runner, runs
  - treated separately
  - no morphology understanding